CookMem Tutorial 5

# Limiting Memory Use

Cap what a context, a library or an input can take



For programs that run plugins, scripts or untrusted input, and services with many tenants

# Why limit memory

1

### Untrusted input

A file or a message that claims gigabytes, such as a decompression bomb.

2

### Third-party code

A plugin or a script that never stops allocating.

3

### Many tenants

One customer or request using far more than its share.

**Without a limit:** swapping, the out-of-memory killer, or a crash.

**With a limit:** that part's allocations fail, and the rest of the program goes on.

# A footprint limit

```
cookmem::SimpleMemContext<> memCtx;
memCtx.setFootprintLimit (1024 * 1024);     // 1 MB

void* block = memCtx.allocate (100 * 1024);
if (block == nullptr)
{
    // Over the limit: handle it; the context is still usable.
}

memCtx.getFootprint ();          // bytes taken from the system
```

*examples/01_footprint_limit.cpp (shortened)*

**nullptr, not a crash:** the context stays usable, and freeing memory makes room again.

**Under a 1 MB limit**

**10**

blocks of 100 KB fit; the 11th returns nullptr

# Containers throw std::bad_alloc

```
memCtx.setFootprintLimit (4 * 1024 * 1024);     // 4 MB
cookmem::Allocator<int> allocator (memCtx);
std::vector<int, cookmem::Allocator<int>> values (allocator);
try
{
    for (int i = 0; ; ++i)
        values.push_back (i);
}
catch (const std::bad_alloc&)
{
    // values is unchanged by the push_back that failed.
}
```

*examples/02_containers_bad_alloc.cpp (shortened)*

1. **std::bad_alloc**
   From Allocator and MemResource.

2. **Strong guarantee**
   push_back leaves the vector unchanged.

3. **Still usable**
   The container and the context go on.

# Untrusted input

### A malicious record




4 bytes of header that lie about the size

```
std::uint32_t length = readLength (record);
char* payload = static_cast<char*> (
    memCtx.allocate (length + std::size_t (1)));
if (payload == nullptr)
    return reject (record);     // over the 16 MB limit
```

*examples/03_untrusted_input.cpp (shortened)*

### With a 16 MB limit

good record, 5 bytes

**accepted**

bad record, claims 3.75 GB

**rejected**

# Libraries that accept an allocator

| Library | Hook | Scope |
|---|---|---|
| **Lua** | `lua_newstate (alloc, ud)` | per interpreter |
| **zlib** | `z_stream: zalloc, zfree, opaque` | per stream |
| **FreeType** | `FT_New_Library (memory, ...)` | per library object |
| **PCRE2** | `pcre2_general_context_create` | per context |
| **libpng** | `png_create_read_struct_2` | per image |
| **SQLite** | `sqlite3_config (SQLITE_CONFIG_MALLOC)` | process-wide |

Check how each library handles a failed allocation: these report it as an error instead of crashing.

# One adapter for a Lua-style hook

```
void* contextAlloc (void* ud, void* ptr,
                    std::size_t osize, std::size_t nsize)
{
    Context* memCtx = static_cast<Context*> (ud);
    if (nsize == 0)
    {
        memCtx->deallocate (ptr);
        return nullptr;
    }
    return memCtx->reallocate (ptr, nsize);   // nullptr when full
}

// With Lua: one interpreter, one context, one limit.
lua_State* L = lua_newstate (contextAlloc, &memCtx);
```

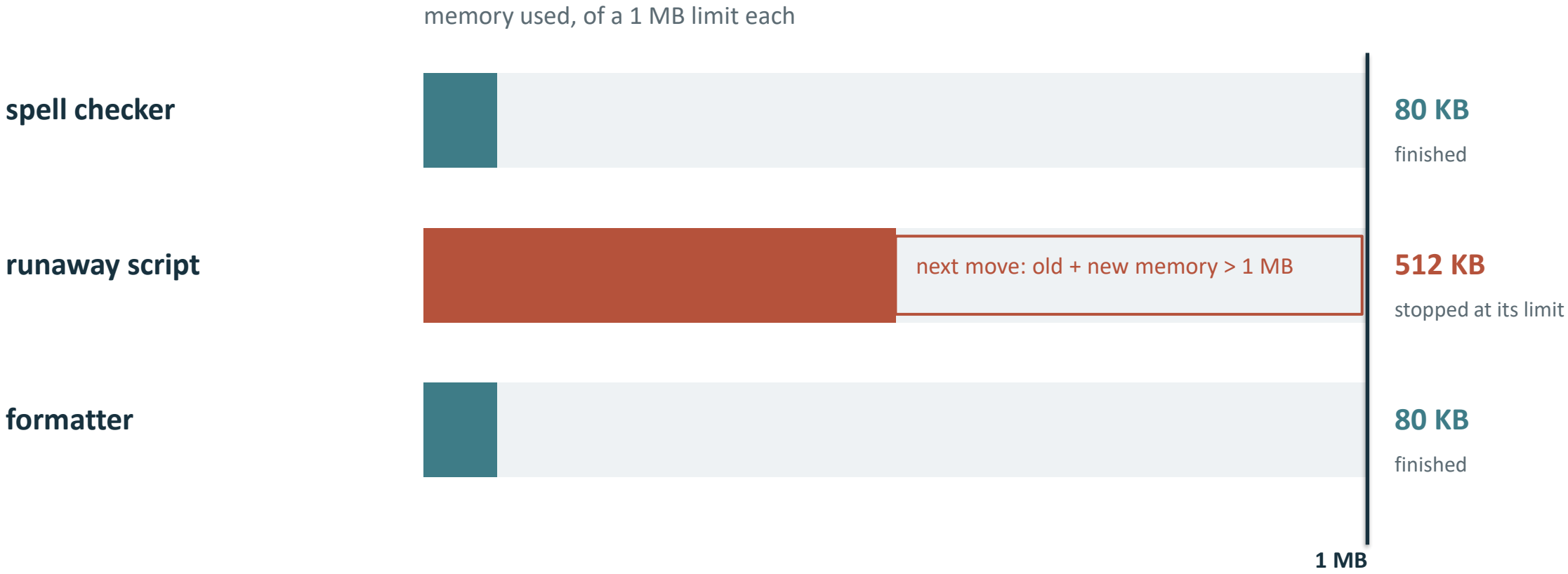
*examples/04_library_hooks.cpp (shortened)*

1. **nsize == 0**
   Free, and return nullptr.

2. **Otherwise**
   reallocate (), which also allocates.

3. **ud**
   The context of this interpreter.

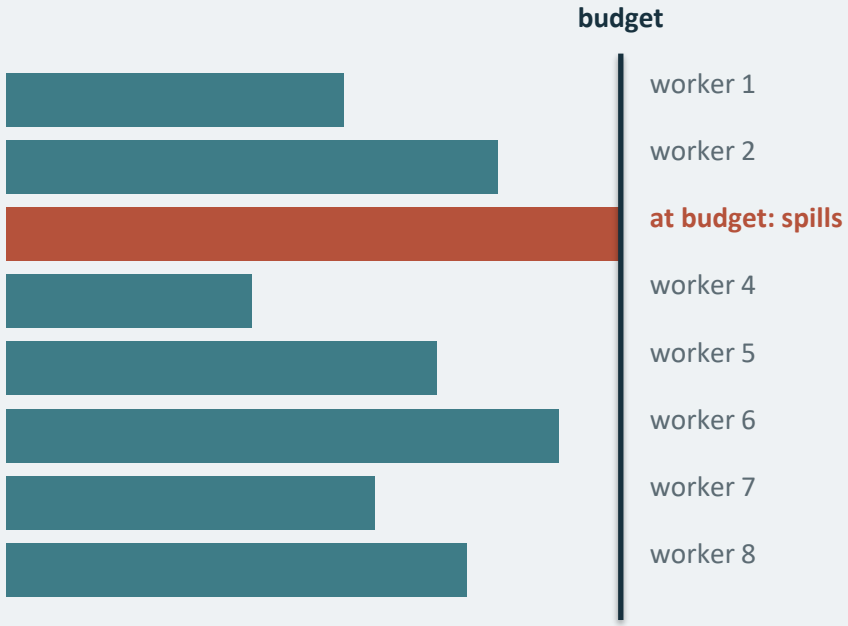
# One budget per plugin




**Isolated budgets:** one plugin reaching its limit does not take memory from the others.

# Budget each worker thread

### Threads x budget < memory




**8 workers x 1 GB budget = 8 GB**

fits in 16 GB, with room for the rest

### Database engines

A memory budget for each sort or hash join, like PostgreSQL's work_mem; over it, spill to disk.

### Parallel builds

Many compile or link jobs at once; a budget per job keeps the build machine out of swap.

### Batch and ETL workers

Each worker processes smaller batches when it reaches its budget.

### Game engines

A fixed budget per subsystem, such as audio or streaming, on consoles with fixed memory.

```
Provider::getThreadContext ().setFootprintLimit (budget);
```

# How big a limit?

1

### Whole segments

At least 64 KB each by default, plus bookkeeping. Set limits well above that.

2

### Growing by moving

Needs old and new memory at once: a growing buffer reaches about half the limit.

3

### Empty segments are released first

Before an allocation fails on the limit, the pool releases segments that became empty.

# Pitfalls to avoid

### Not process isolation

Code that calls malloc or new directly bypasses the limit.

### Libraries must handle nullptr

Lua, zlib and SQLite do; check the ones you use.

### Limits are per context

The process total is every limit plus everything else.

### Dropping a context skips cleanup

Only when the plugin will never touch that memory again.

# Summary

1. **A limit per context**
   Allocations over it fail cleanly: nullptr, or std::bad_alloc from containers.

2. **Untrusted input, plugins and workers**
   Give each its own context and budget; one that runs over fails alone.

3. **Libraries with allocation hooks**
   One small adapter puts Lua, zlib and others on a budget.



Next: Tutorial 6, Replacing malloc and new · github.com/coconut2015/cookmem-tutorials